Ecole Supérieure Privée Technologie & Ingénierie



A.U.2022/2023

Gestion de versions décentralisée

ING TIC-1 / Cours intégré (21h00 Cours/TP)





Dr. Ing. Sawssen JALEL
sawssen.jalel@tek-up.tn

Objectifs de ce cours :

- ✓ Comprendre les principes fondamentaux des VCS et leur importance dans le développement de logiciels.
- ✓ Savoir utiliser Git pour suivre les modifications de code et travailler avec des branches et des révisions.
- ✓ Comprendre comment utiliser Git pour collaborer efficacement avec d'autres développeurs sur un projet.
- ✓ Être capable de résoudre les conflits de fusion et d'éviter les pertes de données.
- ✓ Comprendre comment utiliser GitHub pour héberger des dépôts Git et faciliter la collaboration.

Chapitre 1:
Les bases de Git

Plan



1. Qu’est ce que Git
2. Utilité des systèmes de gestion de versions (SCV)
3. Modèles centralisés et décentralisés des SCV
4. Qu’est ce que GitHub
5. Installation et paramétrage de Git
6. Fonctionnement de base de Git
7. Créer un dépôt Git
8. Modifier un dépôt Git
9. Annuler des actions et consulter l’historique Git

Qu’est ce que Git



- Git est **un système de gestion de version décentralisé open-source**, créé en 2005 par Linus Torvalds, le créateur de Linux.
- Il permet de suivre les modifications apportées à un ensemble de fichiers au fil du temps.
- Les développeurs peuvent travailler sur leur propre branche de développement et fusionner les modifications apportées par les autres, ce qui facilite la collaboration.
- Git permet de résoudre les conflits qui peuvent survenir lors de la fusion de modifications.
- Git est rapide, fiable et peut être utilisé pour gérer des projets de toute taille.
- Git est compatible avec une large gamme d'outils et de services de développement, ce qui en fait un choix populaire pour la gestion de version de code.

Utilité des systèmes de gestion de versions (SCV)

- **Suivi des modifications :** Les SCV enregistrent chaque version et les modifications apportées, ce qui permet aux développeurs de retrouver facilement des versions antérieures du code et de comprendre son évolution.
- **Collaboration :** Les SCV permettent à chaque développeur de travailler sur sa propre branche, ce qui facilite la collaboration et la fusion des modifications apportées.
- **Gestion des conflits :** Les SCV permettent de gérer les conflits qui peuvent survenir lors de la fusion de modifications apportées par différents développeurs.
- **Sauvegarde :** Les SCV offrent une sauvegarde fiable et efficace de l'ensemble du code source et de son historique, ce qui réduit les risques de perte de données.
- **Développement itératif :** Les SCV permettent de travailler de manière itérative en travaillant sur des fonctionnalités spécifiques de manière isolée, ce qui facilite le développement par étapes et réduit les risques de régressions.

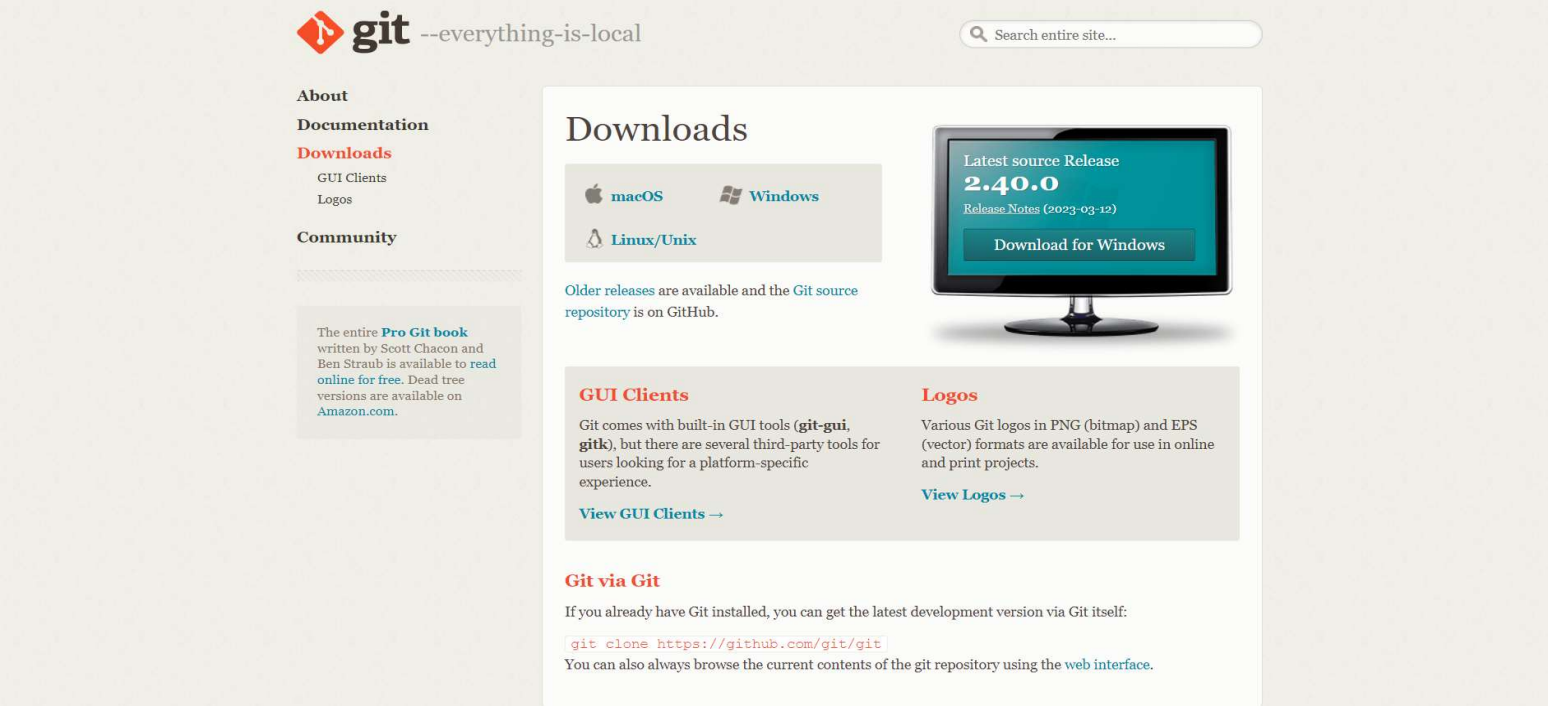
Qu’est ce que GitHub



- GitHub est une plateforme en ligne basée sur Git.
- Il permet la gestion de projet collaborative.
- GitHub offre un hébergement gratuit pour les dépôts Git.
- Il facilite la collaboration entre les développeurs.
- Il fournit des outils de communication tels que les commentaires, les notifications et les discussions.
- GitHub est largement utilisé pour l'hébergement de projets open source.

Pour récapituler , Git est un logiciel de gestion de version tandis que GitHub est un service en ligne d’hébergement de dépôts Git qui fait office de serveur central pour ces dépôts.

Installation de Git



Lien : https://git-scm.com/downloads

Paramétrage de Git

- Une fois Git est installé sur votre système, vous voudrez personnaliser votre environnement Git.
- Vous ne devriez avoir à réaliser ces réglages qu'une seule fois ; ils persisteront lors des mises à jour. Vous pouvez aussi les changer à tout instant en relançant les mêmes commandes.
- Nous allons notamment ici renseigner un nom d'utilisateur et une adresse mail que Git devra utiliser ensuite.
- Pour faire cela, nous allons utiliser notre première commande Git qui est la commande **git config**. Cette commande permet de voir et modifier les variables de configuration qui contrôlent tous les aspects de l'apparence et du comportement de Git.
- L'option **--global** va nous permettre d'indiquer à Git que le nom d'utilisateur et l'adresse mail renseignés doivent être utilisés globalement (c'est-à-dire pour tout projet Git).

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

Paramétrage de Git

- Si vous souhaitez vérifier vos réglages, vous pouvez utiliser la commande git config --list pour lister tous les réglages que Git a pu trouver jusqu'ici :

```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

- Vous pouvez aussi vérifier la valeur effective d'un paramètre particulier en tapant git config <paramètre> :

```
$ git config user.name
John Doe
```

Fonctionnement de base de Git

Démarrer un Dépôt Git :

- Un “dépôt” correspond à la copie et à l’importation de l’ensemble des fichiers d’un projet dans Git.
- Il existe deux façons de créer un dépôt Git :
  - ➢ On peut importer un répertoire déjà existant dans Git ;
  - ➢ On peut cloner un dépôt Git déjà existant.

Fonctionnement de base de Git

La gestion des informations selon Git :

- Git pense les données à la manière d'un flux d'instantanés ou "snapshots". Grosso modo, à chaque fois qu'on va valider ou enregistrer l'état d'un projet dans Git, il va prendre un instantané du contenu de l'espace de travail à ce moment et va enregistrer une référence à cet instantané pour qu'on puisse y accéder par la suite.

- Chaque instantané est stocké dans une base de donnée locale, c'est-à-dire une base de donnée située sur notre propre machine.

- Le fait que l'on dispose de l'historique complet d'un projet localement fait que la grande majorité des opérations de Git peuvent être réalisées localement, c'est-à-dire sans avoir à être connecté à un serveur central distant. Cela rend les opérations beaucoup plus rapides et le travail de manière générale beaucoup plus agréable.

Fonctionnement de base de Git

Les états des fichiers :

- Comment Git fait il pour suivre les modifications sur les fichiers d'un projet ? Pour comprendre cela, il faut savoir qu'un fichier peut avoir deux grands états dans Git : il peut être sous suivi de version ou non suivi.

- Un fichier possède l'état "suivi" si il appartenait au dernier instantané capturé par Git, c'est-à-dire si il est enregistré en base. Tout fichier qui n'appartenait pas au dernier instantané et qui n'a pas été indexé est "non suivi".

Fonctionnement de base de Git

Les états des fichiers :

- Lorsqu'on démarre un dépôt Git en important un répertoire déjà existant depuis notre machine, les fichiers sont au départ tous non suivis. On va donc déjà devoir demander à Git de les indexer et de les valider (les enregistrer en base).
- Lorsqu'on clone un dépôt Git déjà existant, c'est différent puisqu'on copie tout l'historique du projet et donc les fichiers sont tous déjà suivis par défaut.
- Ensuite, chaque fichier suivi peut avoir l'un de ces trois états :
  - Modifié ("modified") ;
  - Indexé ("staged") ;
  - Validé ("committed").

Fonctionnement de base de Git

Les états des fichiers :

- Lors du démarrage d'un dépôt Git à partir d'un dépôt local, on demande à Git de valider l'ensemble des fichiers du projet. Un fichier est "validé" lorsqu'il est stocké dans la base de donnée locale. Lors du clonage d'un dépôt déjà existant, les fichiers sont enregistrés par défaut en base et ils sont donc validés par défaut.

- Ensuite, lorsqu'on va travailler sur notre projet, on va certainement ajouter de nouveaux fichiers ou modifier des fichiers existants. Les fichiers modifiés vont être considérés comme "modifiés" par Git tandis que les nouveaux fichiers vont être "non suivis". Un fichier modifié est considéré comme "modifié" par Git tant qu'il n'a pas été indexé.

Fonctionnement de base de Git

Les états des fichiers :

- On dit qu'on "indexe" un fichier lorsqu'on indique à Git que le fichier modifié ou que le nouveau fichier doit faire partie du prochain instantané dans sa version actuelle.

- Enfin, lorsqu'on demande à Git de prendre l'instantané, c'est-à-dire lorsqu'on lui demande d'enregistrer en base l'état du projet actuel (c'est-à-dire l'ensemble des fichiers indexés et non modifiés), les fichiers faisant partie de l'instantané sont à nouveau considérés comme "validés" et le cycle peut recommencer.

Fonctionnement de base de Git

Les zones de travail :

- Les états de fichiers sont liés à des zones de travail dans Git. En fonction de son état, un fichier va pouvoir apparaitre dans telle ou telle zone de travail.

- Tout projet Git est composé de trois sections :
  - le répertoire de travail (working tree),
  - la zone d'index (staging area) et
  - le répertoire Git (repository).

Fonctionnement de base de Git

Les zones de travail :

- **Le répertoire de travail ou “working tree” :** correspond à une extraction unique (“checkout”) d’une version du projet. Les fichiers sont extraits de la base de données compressée située dans le répertoire Git et sont placés sur le disque afin qu’on puisse les utiliser ou les modifier.
- **La zone d’index ou “staging area” :** correspond à un simple fichier, généralement situé dans le répertoire Git, qui stocke les informations concernant ce qui fera partie du prochain instantané ou du prochain “commit”.
- **Le répertoire Git :** est l’endroit où Git stocke les méta-données et la base de données des objets de votre projet. C’est la partie principale ou le coeur de Git.

Fonctionnement de base de Git

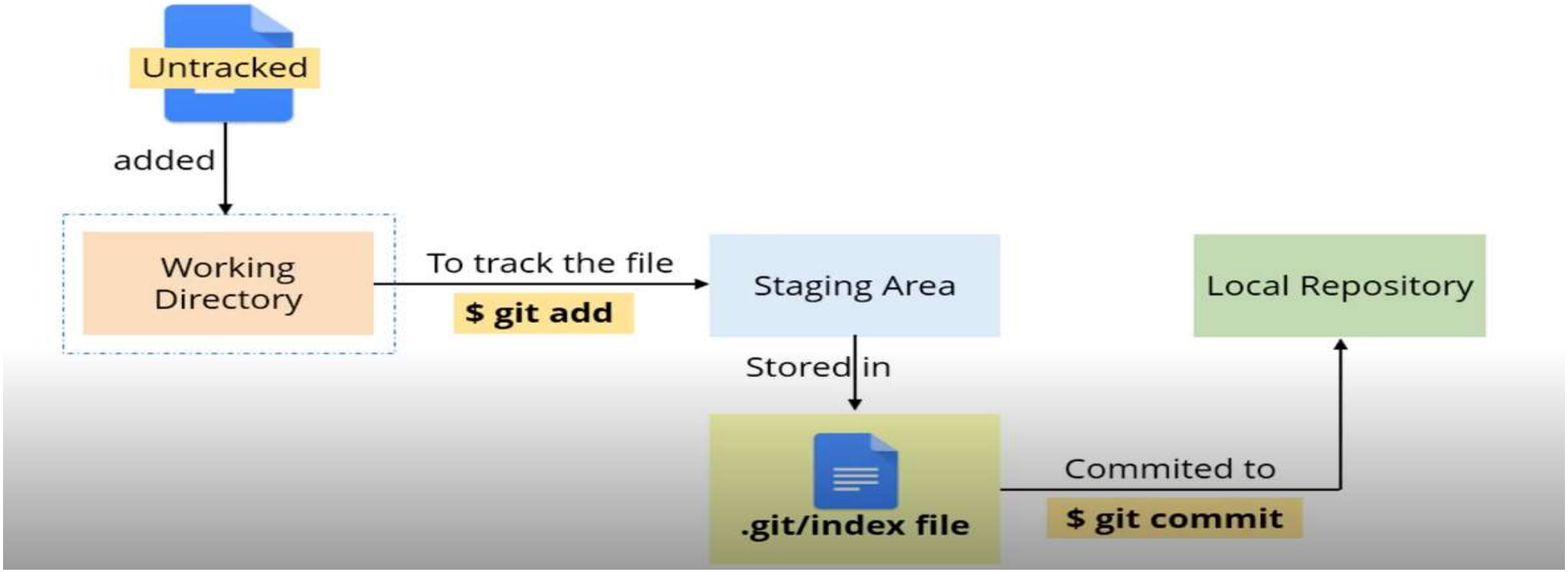
Les zones de travail :

Le processus de travail va ainsi être le suivant :

- Nous allons travailler sur nos fichiers dans le répertoire de travail.
- Lorsqu'on modifie ou crée un fichier, on peut ensuite choisir de l'indexer. Tant qu'un fichier n'est pas indexé, il possède l'état modifié ou est non suivi si c'est un nouveau fichier.
- Dès qu'il est indexé i.e que son nom est ajouté à la zone d'index, il possède l'état indexé.
- Finalement, on va valider ("commit") la version indexée de nos fichiers pour les ajouter au répertoire Git.

Fonctionnement de base de Git

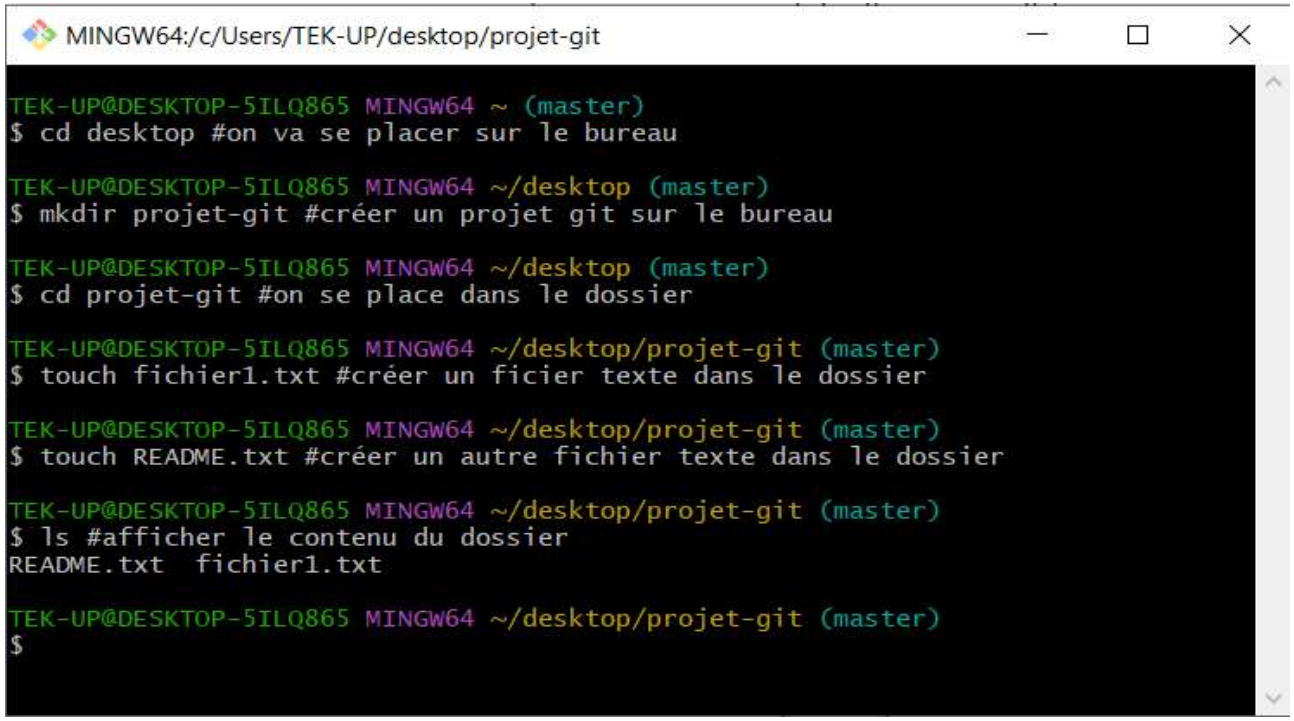
Les zones de travail :



Créer un dépôt Git à partir d'un répertoire existant

- Nous allons créer un répertoire "projet-git" qui se trouve sur le bureau et qui contient deux fichiers texte vides "fichier1.txt" et "README.txt".

- Ce répertoire va nous servir de base pour les exemples qui vont suivre (ce sera le répertoire importé).



- ✓ La commande cd sert à de placer dans un répertoire. Dès qu'on est sur le bureau, on utilise mkdir pour créer un répertoire vide qu'on appelle "projet-git".

- ✓ On se place dans ce répertoire et on crée deux fichiers texte grâce à la commande Bash touch.

- ✓ On utilise enfin ls pour afficher le contenu du répertoire et s'assurer que tout a bien fonctionné.

Créer un dépôt Git à partir d'un répertoire existant

- Pour initialiser un dépôt Git, on utilise ensuite la commande git init comme ci-dessous.

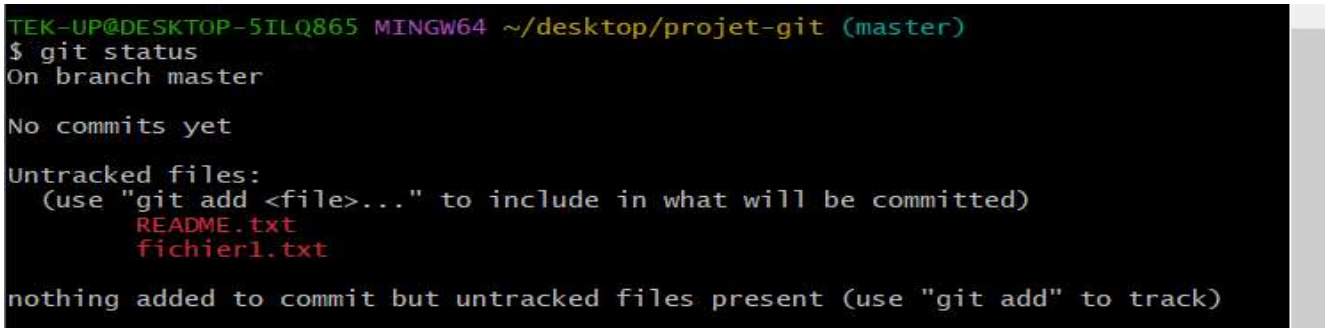
- Cela crée un sous répertoire .git qui contient un ensemble de fichiers qui vont permettre à un dépôt Git de fonctionner.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git init #Initialiser un dépôt Git
Initialized empty Git repository in C:/Users/TEK-UP/Desktop/projet-git/.git/
```

- Lorsqu'on utilise git init, Git nous renvoie un message en nous informant que le dépôt Git a bien été initialisé et qu'il est vide. C'est tout à fait normal puisque nous n'avons encore versionné aucun fichier (nous n'avons ajouté aucun fichier du répertoire en base).

Créer un dépôt Git à partir d'un répertoire existant

- On peut utiliser ici la commande git status pour déterminer l'état des fichiers de notre répertoire. Cette commande est extrêmement utile.



- Ici, git status nous informe que notre projet possède deux fichiers qui ne sont pas sous suivi de version ("untracked") et qui sont les fichiers "README.txt" et "ficiher1.txt".

- Il nous dit aussi qu'aucun fichier n'a été validé ("commit") en base pour le moment ni ajouté pour validation.

- La commande git statuts nous informe également sur la branche sur laquelle on se trouve ("master" ici). Nous reparlerons des branches plus tard.

Créer un dépôt Git à partir d'un répertoire existant

- L'étape suivante va donc ici être d'indexer nos fichiers afin qu'ils puissent ensuite être validés, c'est-à-dire ajoutés en base et qu'on puisse ainsi avoir un premier historique de version.

- Pour indexer des fichiers, on utilise la commande git add. On peut lui passer un nom de fichier pour indexer le fichier en question, le nom d'un répertoire pour indexer tous les fichiers du répertoire d'un coup ou encore un "fileglob" pour ajouter tous les fichiers correspondant au schéma fourni.

- Les fileglobs utilisent les extension de chemin de fichier. Grosso-modo, cela signifie que certains caractères comme * et ? vont posséder une signification spéciale et nous permettre de créer des schémas de correspondances.

- Le caractère * par exemple correspond à n'importe quel caractère. Lorsque j'écris git add *.txt, je demande finalement à Git d'ajouter à l'index tous les fichiers du projet qui possèdent une extension .txt, quelque soit leur nom.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add *.txt #Indexer tous les fichiers texte
```

Créer un dépôt Git à partir d'un répertoire existant

- Si on relance une commande git status, on obtient les informations suivantes :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.txt
        new file:   fichier1.txt
```

- git status nous dit qu'on a maintenant deux nouveaux fichiers ajoutés à l'index.

- La commande git add permet en fait de faire plusieurs choses : elle permet d'indexer des fichiers déjà sous suivi de version et de placer sous suivi des fichiers non suivi (en plus de les indexer).

- Ici, on est certains que nos deux nouveaux fichiers ont bien été ajoutés à l'index puisqu'ils apparaissent dans la section "changes to be committed" ("modifications à valider").

Créer un dépôt Git à partir d'un répertoire existant

- Pour valider ces fichiers et les ajouter en base, nous allons maintenant utiliser la commande git commit comme cela :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m "Version initiale du projet" #valider les fichiers
[master (root-commit) 3119da5] Version initiale du projet
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.txt
 create mode 100644 fichier1.txt
```

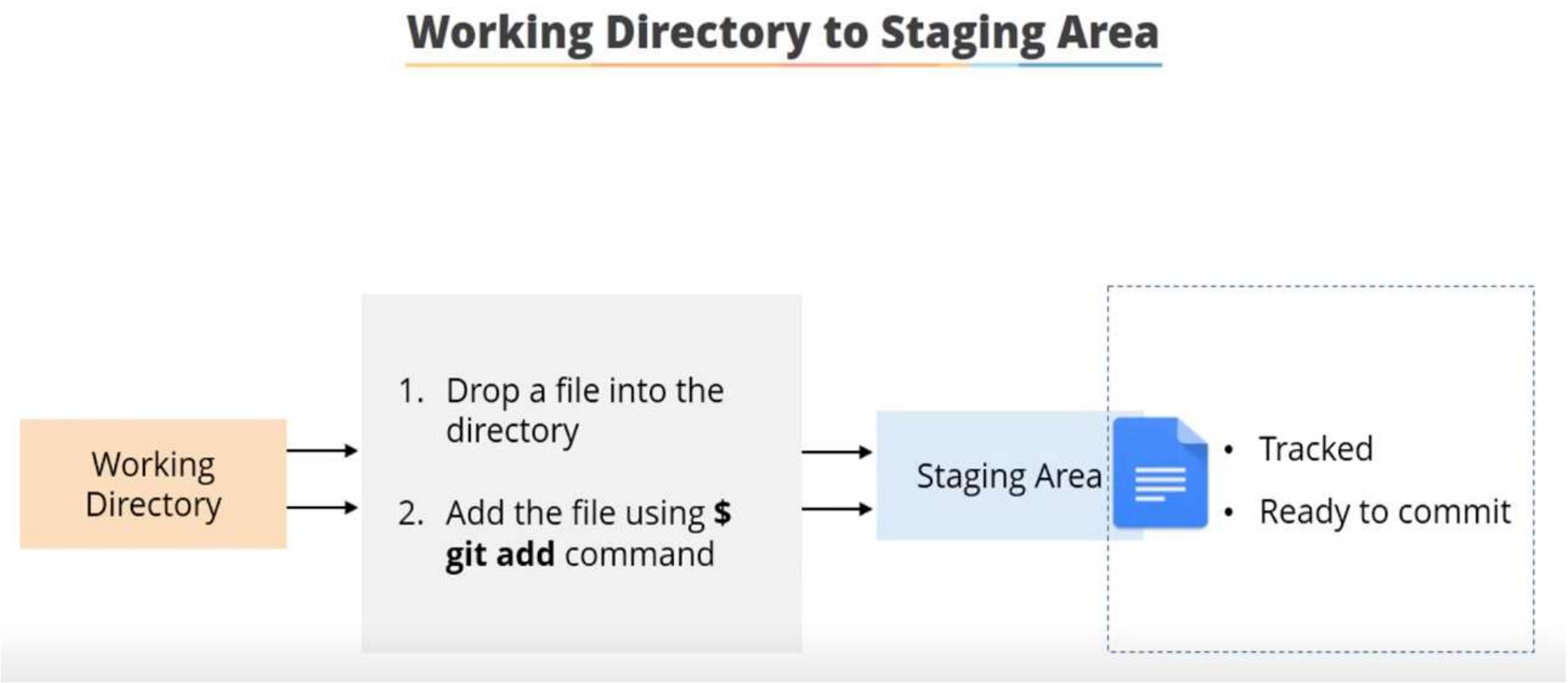
  - On nous informe ici qu'on se situe sur la branche master, qu'il s'agit du premier commit (root-commit) et on nous donne sa somme de contrôle (3119da5) qui permet de l'identifier de manière unique.
  - On nous dit également que deux fichiers ont été modifiés et que 0 lignes ont été ajoutées ou supprimées dans ces fichiers.

- Si on effectue à nouveau un git status, voici le message renvoyé :

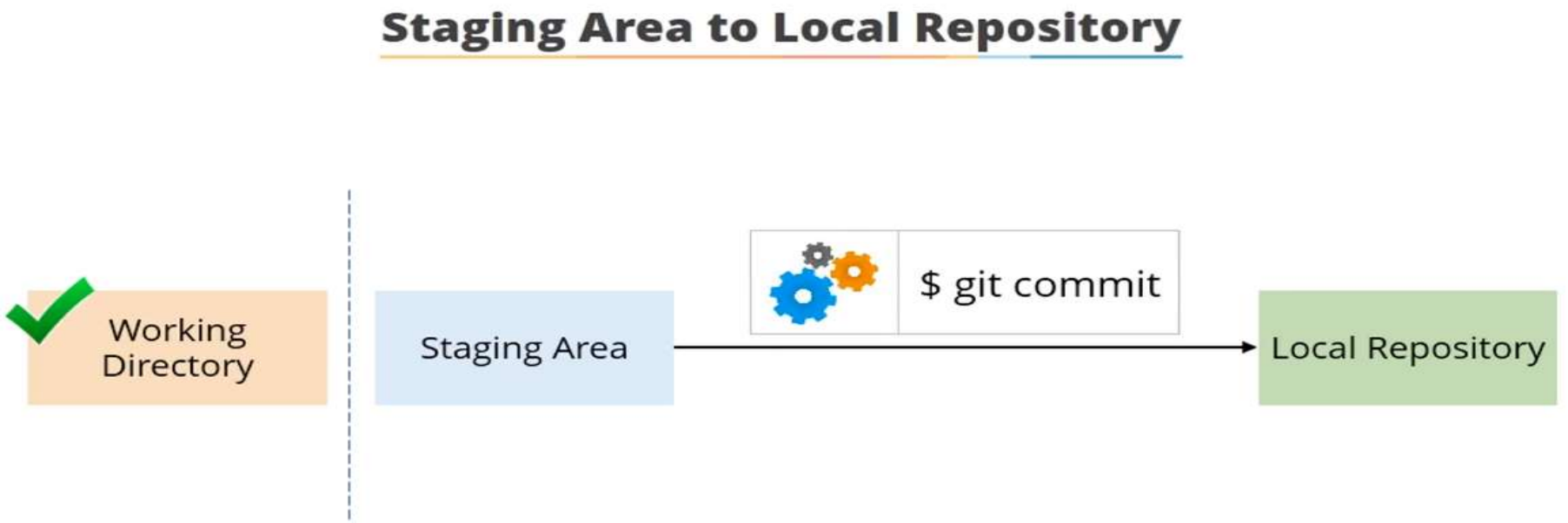
```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

  - Git nous informe désormais qu'il n'y a plus aucun fichier à vider, ce qui signifie que tous les fichiers sont sous suivi de version et sont enregistrés en base et qu'aucune modification n'a été apportée à ces fichiers depuis le dernier commit.

Créer un dépôt Git à partir d'un répertoire existant



Créer un dépôt Git à partir d'un répertoire existant



NOTE

While committing, it is important to add a message string using the **–m flag**. If missed, a default editor opens asking for comments.

Modifier un dépôt Git

Ajouter ou modifier des fichiers dans un projet et actualiser notre dépôt Git

- A chaque fois qu'on souhaite enregistrer une modification de fichier ou un ajout de fichier dans le dépôt Git, on va devoir utiliser les commandes git add et git commit.

- N.B. Le commit (la validation / l'enregistrement en base de données) d'un fichier se basera sur l'état de ce fichier au moment du git add.

- Cela signifie que si vous effectuer une commande git add sur un fichier puis modifiez à nouveau le fichier puis effectuez un git commit, c'est le fichier dans son état au moment du dernier git add qui sera validé et les dernières modifications ne seront donc pas enregistrées dans le dépôt Git.

- Si vous souhaitez enregistrer toujours la dernière version d'un fichier, pensez donc bien toujours à effectuer un git add juste avant un git commit.

- Pour mettre en un coup les fichiers modifiés et déjà sous suivi dans la zone d'index puis pour les valider, vous pouvez également utiliser git commit avec une option -a comme ceci : git commit -a -m. Cela vous dispense d'avoir à taper git add.

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Supprimer un fichier de votre dossier avec la commande Bash rm ne suffira pas pour que Git oublie ce fichier : il apparaitra dans chaque git status dans une section "changes not stages for commit" (modifications qui ne seront pas validées).

- Exemple : Ajouter un fichier test.txt, faire un commit puis le supprimer avec la commande rm.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m "Ajout du fichier test.txt"
[master ebaa3b9] Ajout du fichier test.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test.txt
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ rm test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    test.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Comme vous pouvez le voir, Git continue de suivre le fichier et la suppression simple du fichier ne sera pas validée comme changement par Git.

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Pour supprimer un fichier et l'exclure du suivi de version, nous allons utiliser la commande git rm (et non pas simplement une commande Bash rm).

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git rm test.txt #supprime le fichier du projet et du suivi de version
rm 'test.txt'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    test.txt


TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m "Suppression de test.txt du projet et du suivi"
[master b8f666c] Suppression de test.txt du projet et du suivi
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 test.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

- Pour simplement exclure un fichier du suivi Git mais le conserver dans le projet, on va utiliser la même commande git rm mais avec cette fois-ci une option --cached.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Ajout du fichier fichier2.txt'
[master dac6945] Ajout du fichier fichier2.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 fichier2.txt
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git rm --cached fichier2.txt
rm 'fichier2.txt'
```

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    fichier2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        fichier2.txt
```

Modifier un dépôt Git

Supprimer un fichier d'un projet et / ou l'exclure du suivi de version Git

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Abandon du suivi de fichier2'
[master ffe4962] Abandon du suivi de fichier2
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 fichier2.txt
```

- Ici, le fichier a bien été exclu du suivi Git mais existe toujours dans notre projet. On va ensuite pouvoir modifier ce fichier (lui ajouter du texte par exemple) comme n'importe quel fichier et Git ne se préoccupera pas des modifications.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ echo 'Ajout du texte' >> fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ cat fichier2.txt
Ajout du texte

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status #pour vérifier que git ne suit plus le fichier2.txt
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        fichier2.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Modifier un dépôt Git

Retirer un fichier de la zone d'index de Git

- Le contenu de la zone d'index est ce qui sera proposé lors du prochain commit. Imaginons qu'on ait ajouté un fichier à la zone d'index par erreur. Pour retirer un fichier de l'index, on peut utiliser git reset HEAD nom-du-fichier.

- A la différence de git rm, le fichier continuera d'être suivi par Git. Seulement, le fichier dans sa version actuelle va être retiré de l'index et ne fera donc pas partie du prochain commit.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt
warning: in the working copy of 'fichier2.txt', LF will be replaced by CRLF the next ti
me Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -m 'Sauvegarde fichier2.txt'
[master 121a1f9] Sauvegarde fichier2.txt
 1 file changed, 1 insertion(+)
 create mode 100644 fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ echo 'Modification du fichier 2'>> fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ cat fichier2.txt
Ajout du texte
Modification du fichier 2

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add fichier2.txt
warning: in the working copy of 'fichier2.txt', LF will be replaced by CRLF the next ti
me Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   fichier2.txt
```

Ici, on ajoute à nouveau le fichier fichier2.txt à l'index et on le passe donc sous suivi de version. On valide cela avec git commit puis on modifie le contenu de notre fichier et on ajoute la dernière version de notre fichier à la zone d'index pour qu'il fasse partie du prochain commit.

Modifier un dépôt Git

Retirer un fichier de la zone d'index de Git

- Finalement, on change d'idée et on veut retirer cette version de la zone d'index. On fait cela avec git reset HEAD fichier2.txt.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git reset HEAD fichier2.txt #retirer le fichier de la zone d index
Unstaged changes after reset:
M       fichier2.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   fichier2.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Modifier un dépôt Git

Empêcher l'indexation de certains fichiers dans Git

- Lorsqu'on dispose d'un projet et qu'on souhaite utiliser Git pour effectuer un suivi de version, il est courant qu'on souhaite exclure certains fichiers du suivi de version comme certains fichiers générés automatiquement, des fichiers de configuration, des fichiers sensibles, etc.

- On peut informer Git des fichiers qu'on ne souhaite pas indexer en créant un fichier .gitignore et en ajoutant les différents fichiers qu'on souhaite ignorer. Notez qu'on peut également mentionner des schémas de noms pour exclure tous les fichiers correspondant à ce schéma et qu'on peut même exclure le contenu entier d'un répertoire en écrivant le chemin du répertoire suivi d'un slash.

Modifier un dépôt Git

Empêcher l'indexation de certains fichiers dans Git

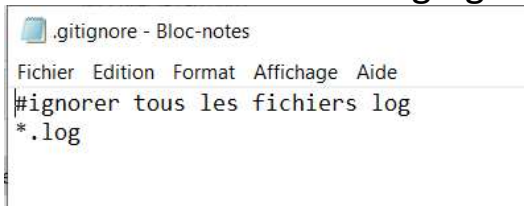
1. Créer un fichier .gitignore :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch .gitignore
```

2. Configurer votre projet

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git config --global core.excludesfile ~/.gitignore_global
```

3. Éditer et tester le fichier .gitignore:




```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   fichier2.txt


TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ touch test.log

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   fichier2.txt
```

Modifier un dépôt Git

Renommer un fichier dans Git

- On peut également renommer un fichier de notre projet depuis Git en utilisant cette fois-ci une commande git mv ancien-nom-fichier nouveau-nom-fichier.

- On peut par exemple renommer notre fichier “README.txt” en “LISEZMOI.txt” de la manière suivante :

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ ls
README.txt  fichier1.txt  fichier2.txt  test.log  test2.log  test3.log

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git mv README.txt LISEZMOI.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ ls
LISEZMOI.txt  fichier1.txt  fichier2.txt  test.log  test2.log  test3.log
```

Modifier un dépôt Git

Renommer un fichier dans Git

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    README.txt -> LISEZMOI.txt
```

Le fichier a bien été renommé dans notre répertoire et le changement est prêt à être validé dans le prochain commit.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit -a -m "renommage du fichie"
[master 3a78e39] renommage du fichie
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename README.txt => LISEZMOI.txt (100%)

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Annuler des actions et consulter l'historique

Consulter l'historique des modifications Git

- La commande git log. Cette commande affiche la liste des commits réalisés du plus récent au plus ancien. Par défaut, chaque commit est affiché avec sa somme de contrôle SHA-1, le nom et l'e-mail de l'auteur, la date et le message du commit.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git log
commit 3a78e39a44dad3a47f2e1db40ab0913aa133f0f2 (HEAD -> master)
Author: Sawssen JALEL <sawssen.jalel@tek-up.tn>
Date:   Sun Apr 9 17:13:29 2023 +0200

    renommage du fichie

commit 9c58c7f98fc7f044277552f7c9e230073c52030a
Author: Sawssen JALEL <sawssen.jalel@tek-up.tn>
Date:   Sun Apr 9 17:06:08 2023 +0200

    Ajout du fichier gitignore
```

- La commande git log supporte également de nombreuses options. Certaines vont pouvoir être très utiles comme par exemple les options -p, --pretty, --since ou --author.

Annuler des actions et consulter l'historique

Consulter l'historique des modifications Git

- Utiliser git log -p permet d'afficher explicitement les différences introduites entre chaque validation.

```
commit 3a78e39a44dad3a47f2e1db40ab0913aa133f0f2 (HEAD -> master)
Author: Sawssen JALEL <sawssen.jalel@tek-up.tn>
Date:   Sun Apr 9 17:13:29 2023 +0200

    renommage du fichie

diff --git a/README.txt b/LISEZMOI.txt
similarity index 100%
rename from README.txt
rename to LISEZMOI.txt
```

- Utiliser git log -n permet d'afficher explicitement les n dernières validations.
- Utiliser git log --online permet d'afficher explicitement chaque commit en une seule ligne.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git log --oneline
3a78e39 (HEAD -> master) renommage du fichie
9c58c7f Ajout du fichier gitignore
121a1f9 Sauvegarde fichier2.txt
ffe4962 Abandon du suivi de fichier2
dac6945 Ajout du fichier fichier2.txt
b8f666c Suppression de test.txt du projet et du suivi
ebaa3b9 Ajout du fichier test.txt
86094b4 version initiale du projet
```

Annuler des actions et consulter l'historique

Ecraser et remplacer un commit

- Pour annuler une validation (un commit), notamment lorsque la validation a été faite en oubliant des fichiers ou sur les mauvaises versions de fichiers, on utilise la commande git commit avec l'option --amend. Cela va pousser un nouveau commit qui va remplacer le précédent en l'écrasant.

1

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git log --oneline
fe974a2 (HEAD -> master) Ajout et modification fichier3
3a78e39 renommage du fichie
9c58c7f Ajout du fichier gitignore
121a1f9 Sauvegarde fichier2.txt
ffe4962 Abandon du suivi de fichier2
dac6945 Ajout du fichier fichier2.txt
b8f666c Suppression de test.txt du projet et du suivi
ebaa3b9 Ajout du fichier test.txt
86094b4 version initiale du projet
```

2

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git mv fichier3.txt fichierTestAnuulationCommit.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    fichier3.txt -> fichierTestAnuulationCommit.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   fichierTestAnuulationCommit.txt
```

3

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git commit --amend -m "Test annulation commit"
[master d58d04d] Test annulation commit
 Date: Mon Apr 10 12:32:19 2023 +0200
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 fichierTestAnuulationCommit.txt
```

4

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/desktop/projet-git (master)
$ git log --oneline
d58d04d (HEAD -> master) Test annulation commit
3a78e39 renommage du fichie
9c58c7f Ajout du fichier gitignore
121a1f9 Sauvegarde fichier2.txt
ffe4962 Abandon du suivi de fichier2
dac6945 Ajout du fichier fichier2.txt
b8f666c Suppression de test.txt du projet et du suivi
ebaa3b9 Ajout du fichier test.txt
86094b4 version initiale du projet
```

Annuler des actions et consulter l'historique

Annuler des modifications apportées à un fichier

- L'un des principaux intérêts d'utiliser un logiciel de gestion de vision est de pouvoir "roll back", c'est-à-dire de pouvoir revenir à un état antérieur enregistré d'un projet.

- Après un commit, on va continuer à travailler sur nos fichiers et à les modifier. Parfois, certaines modifications ne vont pas apporter les comportements espérés et on voudra revenir à l'état du fichier du dernier instantané Git (c'est-à-dire au dernier état enregistré). On va pouvoir faire cela avec la commande générale git checkout -- nom-du-fichier ou la nouvelle commande spécialisée git restore nom-du-fichier.

TP1 : Git/GitHub